Eventos de Dominio e Integración en Arquitectura de Software

Arquitectura Event-Driven con .NET y RabbitMQ

Domain Events • Integration Events • Message Brokers

Juan Carlos De La Cruz

Software Specialist & Instructor

Evolución de las Arquitecturas de Software

- **Monolitos**

  Aplicaciones todo-en-uno, acoplamiento fuerte

- **Microservicios**

  Servicios independientes, escalables

- **Problema**

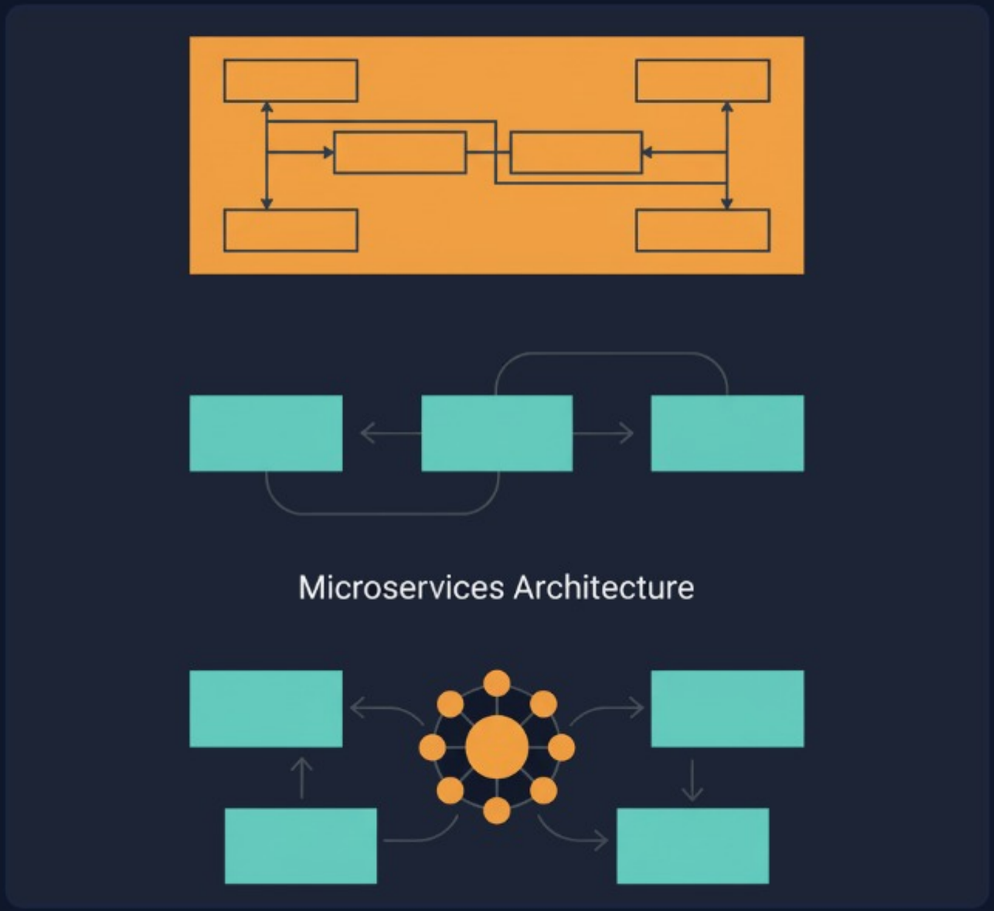
  Comunicación síncrona crea dependencias rígidas

- **Necesidad**

  Desacoplamiento mediante eventos

- **Beneficio**

  Autonomía y resiliencia



Arquitectura Basada en Eventos

Definición: **Patrón donde los servicios reaccionan a eventos**

Comunicación

Comunicación asíncrona y desacoplada

Componentes

Productores, Message Broker, Consumidores

Beneficios

Escalabilidad, tolerancia a fallos, extensibilidad

Casos de uso

E-commerce, fintech, IoT

Eventos en Arquitectura de Software

- Representación de algo que ya ocurrió en el sistema
- Inmutable: no se puede modificar después de crearse
- Incluye: timestamp, datos relevantes, identificadores
- Nombrado en pasado: describe un hecho consumado

Ejemplos

```
PedidoCreado
```

```
PagoAprobado
```

```
UsuarioRegistrado
```

```
InventarioActualizado
```

Fundamentos de DDD

Entidades

Objetos con identidad única

Agregados

Grupo de entidades con raíz

Bounded Context

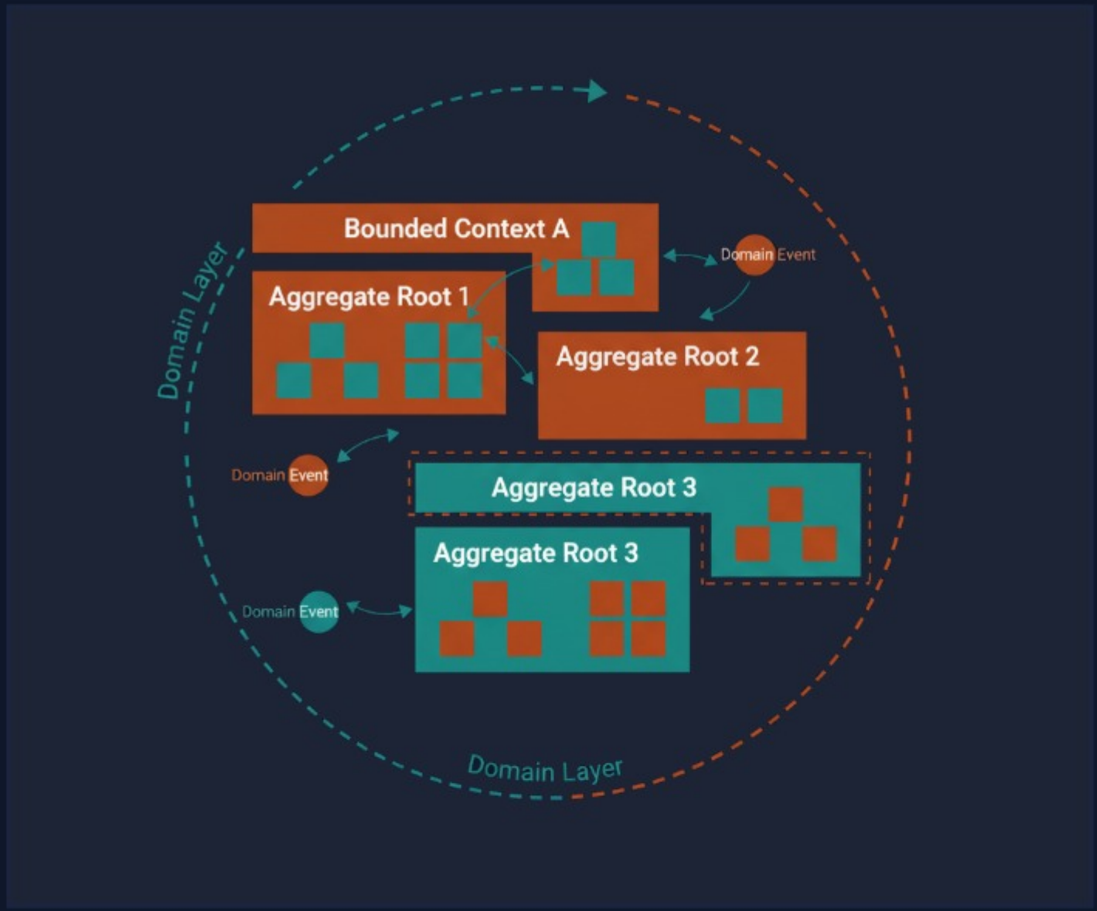
Límites de un modelo de dominio

Domain Events

Eventos que ocurren dentro del dominio

Ubiquitous Language

Lenguaje común del negocio



Eventos de Dominio

DOMAIN EVENTS

Eventos que ocurren dentro de un bounded context

Propósito

Comunicar cambios dentro del mismo dominio

Ámbito

Transaccional, dentro de la misma base de datos

Características

Fuertemente tipados, ricos en contexto de negocio

Cuándo usar

Mantener consistencia dentro del agregado

Caso de Uso: Pedido Creado

Ejemplo Conceptual - Domain Event

1. Usuario crea un pedido en el sistema

2. Agregado "Pedido" dispara: PedidoCreado (Domain Event)

3. **Dentro del mismo contexto se ejecutan:**

   → Actualizar inventario

   → Registrar en auditoría

   → Enviar notificación interna

Todo en una misma transacción

Implementación en C#

.NET

```
// Domain Event

public record PedidoCreado : IDomainEvent
{
    public Guid PedidoId { get; init; }
    public Guid ClienteId { get; init; }
    public decimal Total { get; init; }
    public DateTime FechaCreacion { get; init; }
}
```

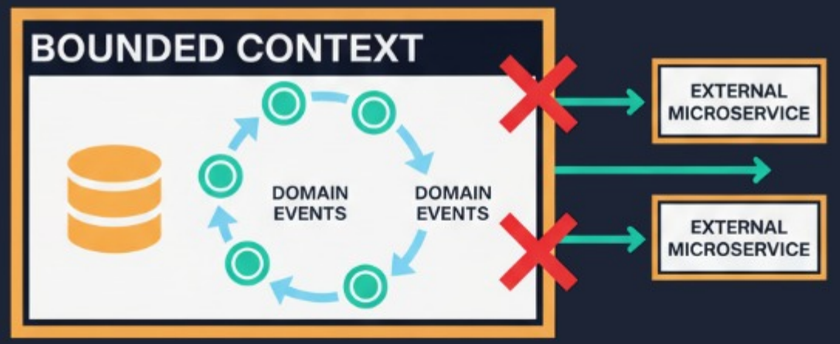
```
// Agregado Pedido

public class Pedido : AggregateRoot
{
    public void Crear(Guid clienteId, decimal total)
    {
        // Lógica de negocio
        AddDomainEvent(new PedidoCreado
        {
            PedidoId = Id,
            ClienteId = clienteId,
            Total = total,
            FechaCreacion = DateTime.UtcNow
        });
    }
}
```

Domain Events en Sistemas Distribuidos

- Solo funcionan dentro del mismo bounded context
- No atraviesan límites de microservicios
- Dependen de la misma transacción de base de datos
- No son adecuados para comunicación entre servicios

Necesitamos otro tipo de evento:

Integration Events



Eventos de Integración

← INTEGRATION EVENTS

Eventos que cruzan bounded contexts

Propósito

Comunicación entre microservicios

Ámbito

Distribuido, a través de message broker

Características

Contrato público, versionados, simples

Cuándo usar

Sincronizar datos entre servicios diferentes

Diferencias Clave

Aspecto	Domain Events	Integration Events
Ámbito	Dentro del bounded context	Entre microservicios
Transacción	Misma transacción DB	Eventual consistency
Acoplamiento	Fuerte (mismo dominio)	Débil (contrato público)
Implementación	In-memory/mediator	Message broker (RabbitMQ)
Visibilidad	Privado al servicio	Público entre servicios
Ejemplo	`PedidoCreado (interno)`	`PedidoConfirmadoEvent`

Flujo de Integration Events

Microservicio A

Orders

publica evento →

Message Broker

RabbitMQ

enruta evento →

Microservicio B

Inventory

Microservicio C

Notifications

Cada servicio mantiene su propia base de datos

RabbitMQ como Message Broker

Message broker open-source basado en AMQP

Exchange

Recibe mensajes y los enruta

Queue

Almacena mensajes para consumidores

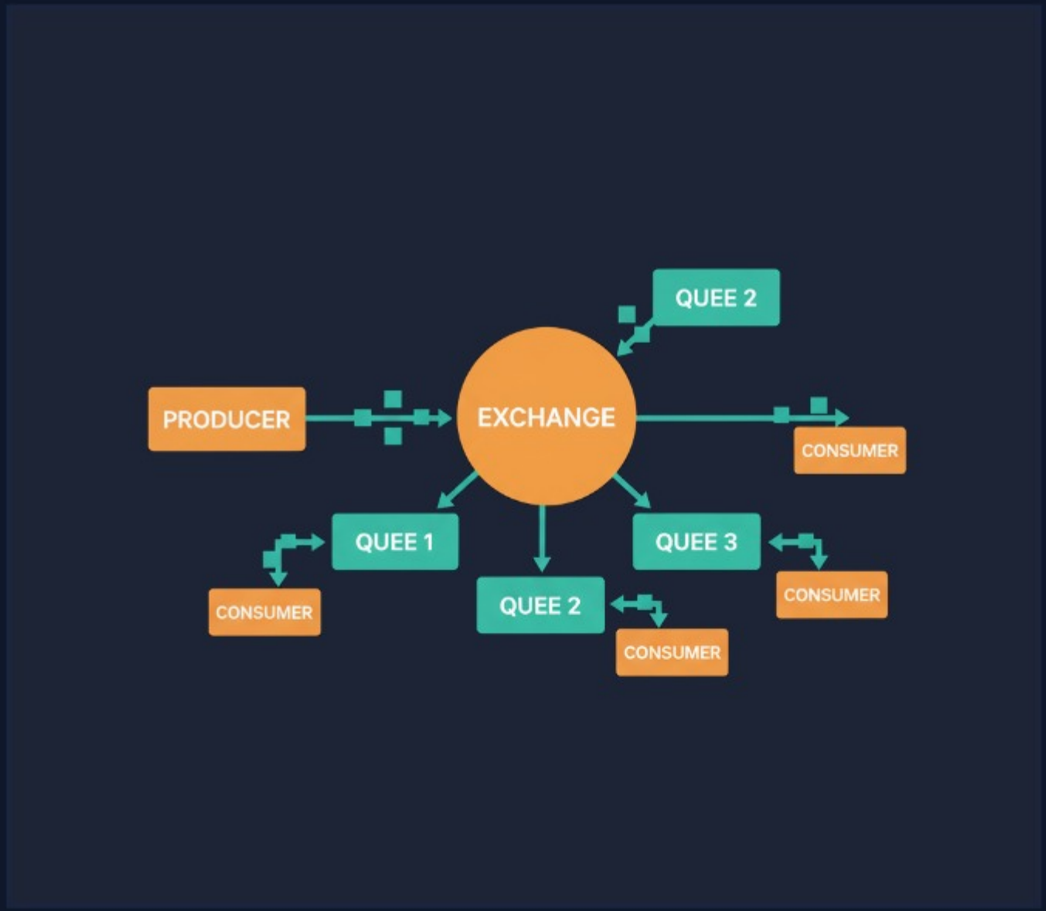
Producer

Publica mensajes

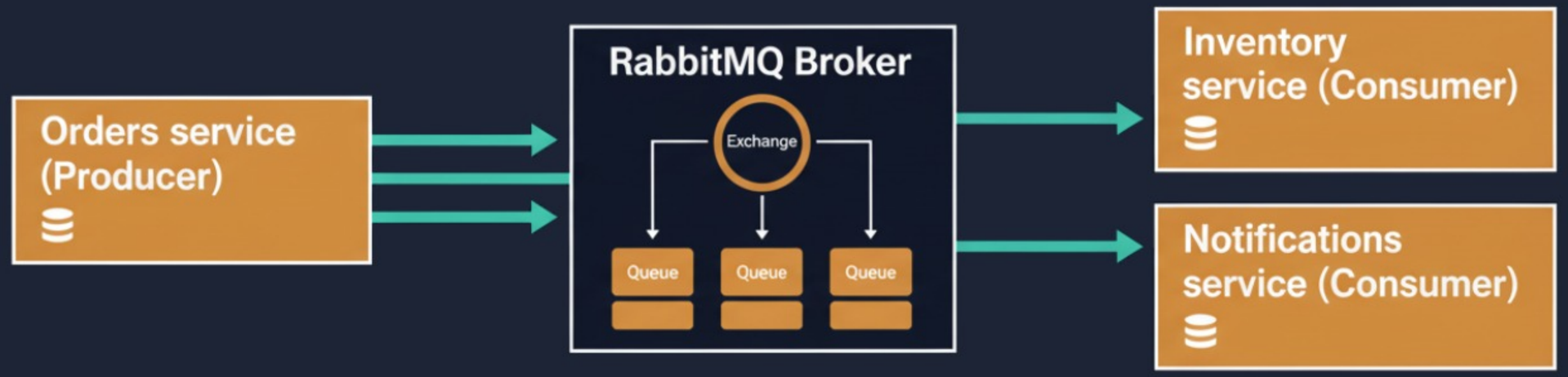
Consumer

Procesa mensajes

Garantiza entrega confiable y escalabilidad



Componentes de la Solución



Publicación de Eventos

PRODUCER

```
// Integration Event

public record PedidoConfirmadoEvent
{
    public Guid PedidoId { get; init; }
    public Guid ClienteId { get; init; }
    public decimal Total { get; init; }
    public List<LineaPedido> Lineas { get; init; }
}
```

```
// Event Bus Publisher

public class RabbitMQEventBus : IEventBus
{
    public async Task PublishAsync<T>(T @event) where T : class
    {
        var json = JsonSerializer.Serialize(@event);
        var body = Encoding.UTF8.GetBytes(json);

        _channel.BasicPublish(
            exchange: "orders_exchange",
            routingKey: typeof(T).Name,
            body: body);
    }
}
```

Consumo de Eventos

CONSUMER

```
// Event Handler

            public class PedidoConfirmadoHandler
     : IEventHandler<PedidoConfirmadoEvent>
{
                public async Task HandleAsync(
          PedidoConfirmadoEvent @event)
     {
          // Actualizar inventario
               await _inventoryService
               .ReservarProductosAsync(@event.Lineas);

          // Lógica de negocio del servicio
          _logger.LogInformation(
                                        "Pedido {PedidoId} procesado",
               @event.PedidoId);
     }
}
```

```
// RabbitMQ Consumer

_channel.BasicConsume(
     queue: "inventory_queue",
     autoAck: false,
     consumer: consumer);
```

Del Domain Event al Integration Event

Flujo Completo End-to-End

1. Usuario crea pedido → Comando
2. Agregado Pedido dispara PedidoCreado (Domain Event)
3. Handler procesa validaciones internas
4. Se publica PedidoConfirmadoEvent (Integration Event)
5. RabbitMQ distribuye a múltiples servicios
6. Inventory service actualiza stock
7. Notification service envía email

Patrones y Recomendaciones

Event Versioning
Usar versionado semántico (v1, v2)

Idempotencia
Handlers deben ser idempotentes

Event Naming
Usar pasado y lenguaje de negocio

Event Contracts
Mantener compatibilidad hacia atrás

Error Handling
Dead letter queues para fallos

Monitoring
Observabilidad de flujos de eventos

Errores a Evitar

- Usar Domain Events para comunicación entre servicios
- Eventos con demasiada información sensible
- No versionar contratos de Integration Events
- Acoplamiento temporal (esperar respuesta síncrona)
- No implementar idempotencia en consumers
- Eventos que representan comandos (usar imperativo)

Resumen y Próximos Pasos

Domain Events: dentro del bounded context, transaccionale

Integration Events: entre microservicios, eventual consistency

RabbitMQ: broker confiable para arquitecturas distribuidas

Desacoplamiento: clave para sistemas escalables

Próximos pasos

Implementar patterns como Saga, Outbox

Recursos

- → Documentación de .NET
- → RabbitMQ tutorials
- → Libros de DDD